
Supplementary information

**Optimizing quantum annealing schedules
with Monte Carlo tree search enhanced
with neural networks**

In the format provided by the
authors and unedited

Optimizing Quantum Annealing Schedules: From Monte Carlo Tree Search to QuantumZero

Yu-Qin Chen,¹ Yu Chen,² Chee-Kong Lee,¹ Shengyu Zhang,¹ and Chang-Yu Hsieh^{1,*}

¹Tencent Quantum Laboratory, Tencent, Shenzhen, Guangdong, China, 518057

²The Chinese University of Hong Kong, Hong Kong

I. SUPPLEMENTARY MATERIAL

A. Stochastic Descent

We use stochastic descent (SD) algorithm to sample the AQC ground energy landscape minima with local optimal evolution schedules as the benchmark of the results acquired using MCTS. SD is a simple algorithm for schedule optimization in discretized search spaces. The algorithm start from a randomly generated initial schedule and perform local field update consistently. The neighbor schedule is accepted $\mathbf{x} \rightarrow \mathbf{x}'$ if it is better than the current one: $\langle \psi(T)^{\mathbf{x}'} | H_{\text{final}} | \psi(T)^{\mathbf{x}'} \rangle < \langle \psi(T)^{\mathbf{x}} | H_{\text{final}} | \psi(T)^{\mathbf{x}} \rangle$. The algorithm stops after a given number of iterations or when there is no better solution when looking up all the neighbors. The obtained schedule is a local minimum respect to local updates. In the main text we perform SD multiples times with different initial random schedules for the same search space.

B. AQC evolution schedule design by Reinforcement learning

Reinforcement Learning (RL) has been a very useful approach for the automation of complex tasks in recent years. We briefly introduce all RL algorithms, benchmarked against Monte Carlo Tree Search and newly proposed Quantum Zero algorithms discussed in this work. Noting that all RL algorithms can be formulated as a Markov Decision Process (MDP), we first introduce the common framework before discussing specific details of each RL algorithm.

For the automated design of annealing schedules, the MDP is given by ,

1. Observable space S . Environment state $s_t \in S$ at a given timestep t with a duration $t \in [0, T]$. Here $s_t = \mathbf{x}$ is a vector of path parameters.
2. Action space A from which an agent picks an action $a_t \in A$ and applies it to a state to get s_{t+1} at each time step t . Action space here is $\{-l, -l + \Delta, l - \Delta, l\}$, where $\pm l$ are the upper and lower bound set for the amplitudes of each frequency component with Δ as the discretized interval.
3. A scalar reward of 1 for the annealed energy satisfying $|E(T) - E_{\text{target}}| \ll \epsilon$ with ϵ a given parameter, and -1 otherwise. $E(T)$ is the environment's feedback, taken as the expectation value of $\langle \psi_{\mathbf{x}}(T) | H_{\text{final}} | \psi_{\mathbf{x}}(T) \rangle$.

Deep Q-Network (DQN) DQN [1, 2] algorithm combines RL with a deep neural network to learn a complex state-action relation in order to accomplish complex tasks. DQN was the first RL algorithm to demonstrate superhuman performance in an Atari game. DQN overcomes unstable learning for nonlinear function approximators such as neural networks by using two techniques: experience replay and target network. Experience replay stores past experiences including state transitions, rewards and actions. These experiences are organized in mini-batches when training neural networks. The mini-batches reduce correlations between experiences used in updating deep neural networks. Target-network technique fixes parameters of a target function and replaces them with the latest network at regular intervals. The DQN network takes states $\{s_t\}$ as an input, and outputs a Q-value for each action, the target Q-value:

$$\hat{Q}_k(s_t, a_t) \leftarrow r_{t+1} + \gamma \max_{\alpha \in A} \hat{Q}_{k-1}(s_{t+1}, \alpha, \theta) \quad (1)$$

The goal for a DQN agent is to maximize expectation of its perceived reward by learning from past examples and formulate an optimal policy. We use the OpenAI Baselines [3, 4] to train DQN agents to design the annealing path following a Markov

* kimhsieh@tencent.com

Decision Process. In the subsection E of Result (in the main text), we choose discount factor $\gamma = 0.99$, neural network of two dense layers $\{64, 64\}$, learning rate $lr = 0.001$, final value of random action probability 0.01 for the DQN model.

Advantage Actor-Critic (A2C) Actor-Critics [5] aim to take advantage of all the good stuff from both value-based RL and policy-based RL by efficiently learns an approximation for both action policy and value functions. A2C framework contains two networks. One of them (actor network) is to produce the best action for a given state. The second network (critic network) learns the advantage value of taking an action as shown in Eq:2:

$$A(s_t, a_t) \leftarrow r_{t+1} + \gamma V_v(s_{t+1}) - V_v(s_t) \quad (2)$$

The goal of an A2C agent is also to maximize the expectation of its perceived reward by learning from known examples. We use the OpenAI Baselines [3, 4] to train A2C agents to design annealing paths. In the subsection E of Result (in the main text), we choose discount factor $\gamma = 0.99$, neural network of two dense layer $\{64, 64\}$, and a learning rate $lr = 0.001$.

Proximal policy optimization (PPO) For policy-based RL, when using gradient descent to optimize a policy objective function, the policy is usually hard to be properly updated leading to gradients vanishing or exploding. PPO [6] tries to compute an update that ensuring the deviation from the previous policy relatively mild. It makes updated policy lying within a trust region and avoids additional overhead to the optimization problem by incorporating a constraint inside the objective function as a penalty. In the PPO framework, the inaccuracy brought by occasional violations of the constraints is generally mild, and the computation is much simpler. We use the OpenAI Baselines [3, 4] to train PPO agents to design the annealing paths. In the subsection E of Result (in the main text), we choose discount factor $\gamma = 0.99$, neural network of two dense layer $\{64, 64\}$.

C. Analysis on the transferability of optimal pulses across 3-SAT problems

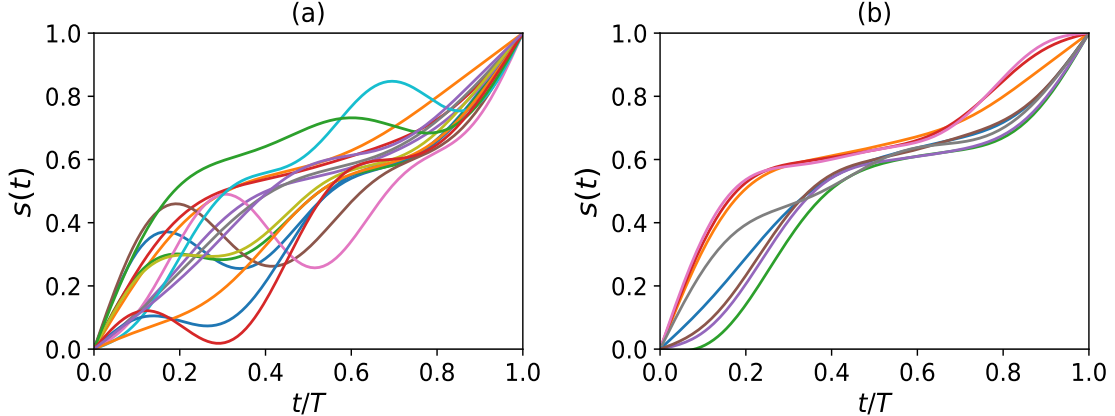


Figure 1: (a) Optimal schedules for different 3-SAT instances ($m/n = 3$) in training dataset at $T = 40$. (b) Top-8 average optimal schedules for 3-SAT instances ($m/n = 3$) in training dataset at $T = 40$.

In the main text, we discuss the transferability of 'optimal' pulses across 3-SAT problems. When $T = 100$ is large, the transferability is good as adiabatic evolution is likely to dominate when the annealing schedule progress slowly enough (and any paths seem to work well). However, as the annealing time budget is reduced, such as $T = 40$ in the main text, directly transfer either an optimal schedule from a random instance or an average optimal schedule from a training dataset does not perform as well. In this non-adiabatic regime, Qzero algorithm trained with the proper transfer learning manifests its superiority. Clearly, different 3-SAT instances (even with the same $m/n = 3$ ratio) possess their own uniqueness such that a direct transfer of an optimal path from another similar problem may not work when the annealing time is not sufficiently long. This point is further analyzed in Fig. 1(a), in which we present some of the optimal schedules for different 3-SAT instances used for transfer learning in the main text. In Fig. 1(b), we show the top-8 average optimal schedules of the training instances. Those average optimal schedules are 'smoother' than the single optimal ones, and they successfully capture some common features across many instances in order to attain a better transferability on average. For instance, all considered 3-SAT problems tend to exhibit minimal gaps cluster around the same point (recalled dimensionless time unit) along the path as shown in Fig.7(c) in the main text. Nevertheless, it is clear that these smooth paths usually deviate from the single optimal ones as shown in Fig. 1(a).

D. Analysis on the Learning efficiency with system size

In the main text we compare the learning efficiencies of MCTS, Qzero-nopre, Qzero-pre and some other common RL algorithms like PPO for 3-SAT problems with the system size $n = 7$. More precisely, we examine the convergence efficiencies for training these different RL algorithms. In order to study the scaling behaviors of those algorithms, here we present the convergence efficiencies of 3-SAT problems with system size $n = 7, 9, 11$ in Fig. 2(a). Since, according to the results presented in the main text, PPO performs best among the other tested RL algorithms, here we only compare Qzero-nopre, Qzero-pre with PPO. As the system size grows, the resource consumption for the learning increases for all algorithms, but Qzero and Qzerop increasingly outperform PPO.

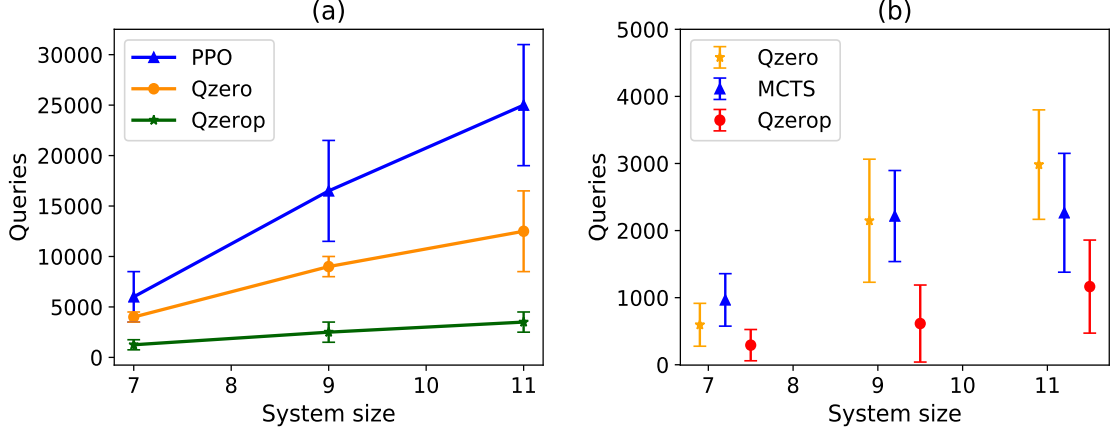


Figure 2: (a) Convergence efficiencies (queries) of Qzero (Qzero-nopre), Qzerop (Qzero-pre) and PPO, on 3-SAT problems with system size $n = 7, 9, 11$ under evolution time $T = 70, 100, 300$ respectively. We test four different 3-SAT instances of $m/n = 3$ for each system size and each algorithms is repeatedly learned for 10 times. (b) Learning efficiencies (queries) of MCTS, Qzero (Qzero-nopre), Qzerop (Qzero-pre) needed first-solution-time that satisfying $F > 0.99, F > 0.99, F > 0.97$ for 3-SAT problems with system size $n = 7$ (under $T = 70$), $n = 9$ (under $T = 100$), $n = 11$ (under $T = 300$) respectively. Each algorithm is repeatedly learned for 30 times.

Next, we also investigate the search efficiency between MCTS and Qzero-pre (enhanced with the transfer learning). For this analysis, we count the number of queries required for the first-solution-time. In other words, we stop the algorithms as soon as they find a solution that certain criterion. More precisely, in Fig. 2(b), the first-solution-time is defined as the first time reaching a solution that satisfying $F > 0.99, F > 0.99, F > 0.97$ for 3-SAT problems with system size $n = 7$ (under $T = 70$), $n = 9$ (under $T = 100$), $n = 11$ (under $T = 300$), respectively. As shown, when the system size scales up, Qzerop consistently outperform pure MCTS.

E. Analysis on the learning efficiency with the size for the state space

In the main text, when running the simulation experiments in the frequency domain, we always truncate the frequency at $M = 5$. Here we add more frequency components to investigate how the algorithms behave under the new simulation setting. As the problem does not really change, merely scaling up the state space only make sense if we raise the bar for a success optimization with a higher fidelity. Since Qzero is built upon MCTS, here we only compare the learning efficiency of MCTS and PPO with respect to the scaling of the size for the state space.

As shown in Fig. 3, MCTS consistently outperform PPO as the state size scales up from 20^5 to 20^8 .

F. Inspecting the MCTS solutions in closer details: Grover's search

For some problem, such as the example of Grover's search, there exists analytically derived non-linear annealing schedules that obey adiabaticity. It will be interesting to analyze how paths proposed by MCTS differs from these theoretically motivated path designs.

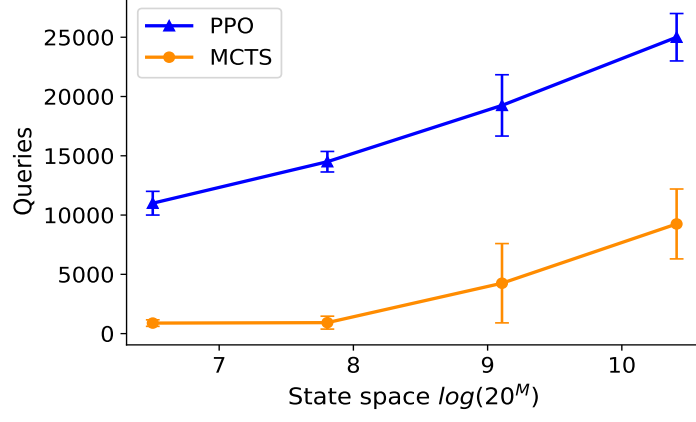


Figure 3: Number of queries needed by MCTS and PPO to converge to fidelity of 0.98 different state spaces with frequency truncation $M = 5, 6, 7, 8$, thus state spaces $20^5, 20^6, 20^7, 20^8$.

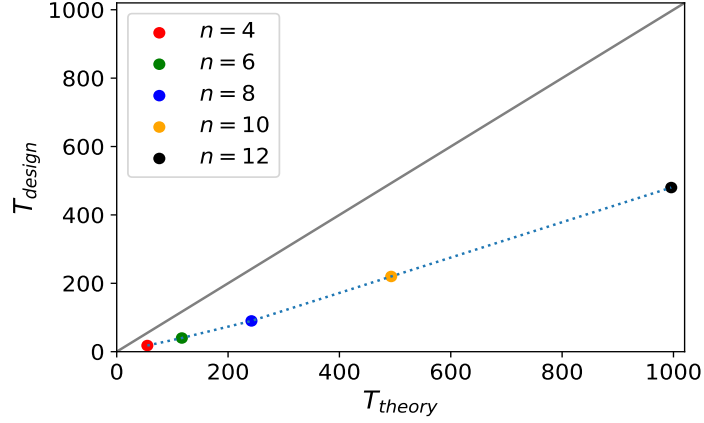


Figure 4: The evolution time needed by theoretical non-linear path and MCTS-based RL designed evolution schedules for system size of $n = 4, 6, 8, 10, 12$.

The problem Hamiltonian of Grover's problem for the adiabatic quantum computation is $H_{final} = \mathbb{I} - |b\rangle\langle b|$, where $|b\rangle$ is a product state in Pauli-Z basis that encodes the search target. The initial trivial Hamiltonian $H_{init} = \mathbb{I} - |\psi_0\rangle\langle\psi_0|$, where $|\psi_0\rangle$ is the product in Pauli-X basis with all n eigenvalues that equals to 1. Given in Ref. [7], the theoretically proposed non-linear path reads

$$s(t) = \frac{1}{2} + \frac{1}{2\sqrt{N-1}} \tan \left[2t\epsilon\sqrt{N-1}/N + \arctan \sqrt{N-1} \right], \quad (3)$$

where $N = 2^n$ is the dimension of corresponding Hilbert space, ϵ^2 denotes the final infidelity $\epsilon^2 = 1 - F$. For the theoretically derived path, it is known that the minimally required evolution time for a given ϵ scales as,

$$T_{theory} = \frac{1}{\epsilon} \frac{N}{\sqrt{N-1}} \arctan \sqrt{N-1}. \quad (4)$$

As we can see from Fig. 4, to realize the same fidelity $F = 0.99$, the annealing schedules designed by MCTS entails much shorter evolution time compared with the time needed by theoretically proposed non-linear path.

We further present the exact diagonalization of energy spectrum for the instantaneous Hamiltonian $H = (1 - s(t))H_{init} + s(t)H_{final}$ along the adiabatic path, as well as the evolution of the expectation value of the instantaneous Hamiltonian with respect to the instantaneous state $\langle\psi_t|H|\psi_t\rangle$ in Fig. 5 for various system size of $n = 4, 6, 8, 10$ for the evolution time $T = 18, 50, 90, 200$ respectively. As clearly revealed in Fig. 5, the accelerated paths (designed by MCTS) clearly invoke non-adiabatic transitions with the expected energies lying above the instantaneous ground-state energies.

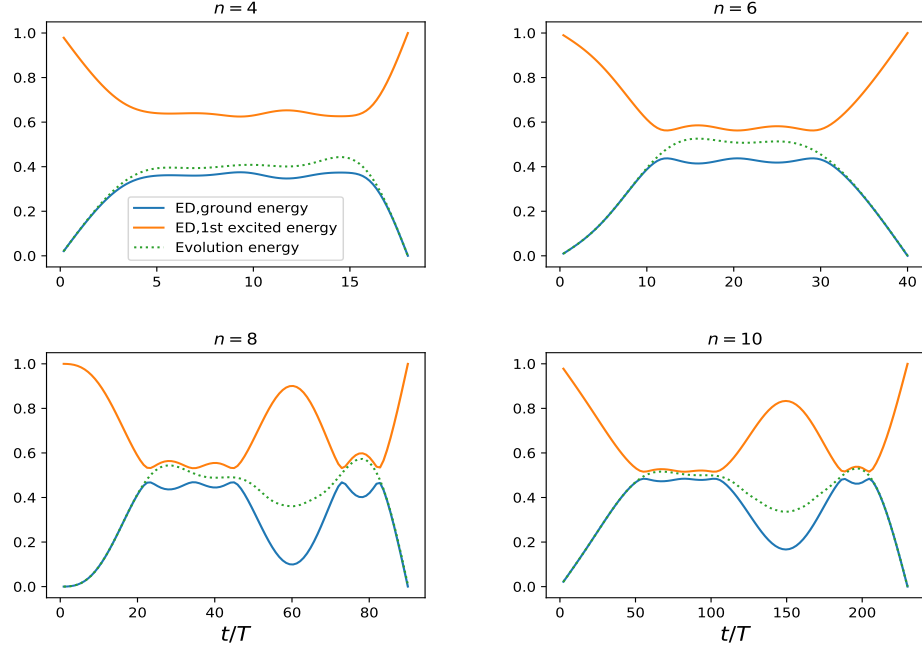


Figure 5: The energy spectrum of instantaneous Hamiltonian during the evolution process for $n = 4, 6, 8, 10$.

G. Quantum annealing simulation in noisy environment

In practice, the adiabatic device usually works in a noisy environment. Here we study the influence of noise on the finally acquired simulation fidelity by adding noises on the rewards during the training of the algorithms.

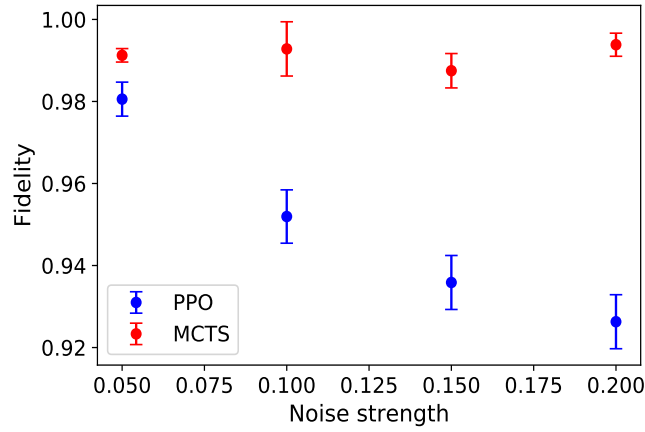


Figure 6: The finally fidelity acquired by algorithms change along with noise strength.

In Fig. 6, we present the the final fidelities acquired by MCST and PPO under the influence of weak-to-moderate noise levels for 3-SAT problem of system size $n = 7$ and evolution time $T = 70$. As shown in the figure, the path designed by MCTS manifests better noise resilience than that by PPO.

H. Performing MCTS in the Quantum Circuit Model

In the main text, we discuss a quantum annealer operating as an analogue device. When a quantum annealing process following a given schedule $s(t)$, it can be understood as follows. The system is initialized in the ground state of a simple Hamiltonian,

and let it evolve for a total annealing time T under the action of $H(s(t)) = (1 - s(t))H_{init} + s(t)H_{final}$. The corresponding evolution operator:

$$U(t, 0) = \mathcal{T} \exp \left(-\frac{i}{\hbar} \int_0^t dt' H(s(t')) \right)$$

where $\mathcal{T} \exp$ denotes the time-ordered exponential. Quantum annealing with a smooth schedule can be easily discretized into a digital version. $s(t)$ can be approximated with K values $s_1, \dots, s_K$ corresponding to evolution times $\Delta t_1, \dots, \Delta t_K$, with $s_j \in (0, 1]$ and $\sum_{j=1}^K \Delta t_j = T$. The evolution operator $U(T, 0)$ then reads

$$U(T, 0) \Longrightarrow U_{\text{step}} = \prod_{j=1}^{\leftarrow K} e^{-\frac{i}{\hbar} H(s_j) \Delta t_j} \quad (5)$$

where the arrow $\leftarrow$ denotes a time-ordered product. A further digitalization step is to perform a Trotter splitting of the term $e^{-\frac{i}{\hbar} H(s_j) \Delta t_j}$. For instance, the lowest-order Trotter splitting

$$e^{-\frac{i}{\hbar} H(s_j) \Delta t_j} \simeq e^{-i\beta_j H_{init}} e^{-i\gamma_j H_{final}} + O((\Delta t_j)^2) \quad (6)$$

with $\gamma_j = s_j \frac{\Delta t_j}{\hbar}$, $\beta_j = (1 - s_j) \frac{\Delta t_j}{\hbar}$ leads to an approximated evolution operator of the form,

$$U(T, 0) \approx U_{\text{digit}}(\gamma, \beta) = U(\gamma_K, \beta_K) \cdots U(\gamma_1, \beta_1) \quad (7)$$

with $U(\gamma_j, \beta_j) \equiv U_j = e^{-i\beta_j H_{init}} e^{-i\gamma_j H_{final}}$. The parameters satisfy $\sum_{j=1}^K (\gamma_j + \beta_j) = \frac{T}{\hbar}$. Hence, one can easily use MCTS to design annealing schedule then perform corresponding unitary transformation to a quantum circuit.

Next, we note the proposed MCTS method can be adapted to the popular Quantum Approximate Optimization Algorithm (QAOA) method [8–10]. QAOA is a hybrid quantum-classical algorithm that combines state preparation in quantum circuits with classical optimization of the circuit parameters to solve the kind of combinatorial optimizations considered in this work. A QAOA circuit (with depth P) alternates the application of H_{init} and H_{final} to prepare a variational quantum state for P times,

$$|\psi_P(\gamma, \beta)\rangle = U(\gamma_P, \beta_P) \cdots U(\gamma_1, \beta_1) |\psi_0\rangle, \quad (8)$$

where $|\psi_0\rangle$ is a chosen initialization. Obviously, the QAOA circuit is analogous to a digitized quantum annealing. When depth P is sufficiently deep, the parameter γ, β of QAOA circuit can be directly taken from a correspondingly digitized quantum annealing process, as shown in Eq. 7; otherwise, these parameters should be obtained by optimizing the cost function $E_P(\gamma, \beta) = \langle \psi_P(\gamma, \beta) | H_{final} | \psi_P(\gamma, \beta) \rangle$ with respect to the parameters. By drawing the analogy between QAOA (with sufficiently long P -depth) and digitized quantum annealing, the proposed MCTS approaches may be easily applied to suggest QAOA parameters for initialization. When P -depth is shallow, one may even directly discretize the search space for (γ, β) and perform MCTS on it.

I. MCTS-designed annealing schedules in the time-frequency domain

In this section, we study the annealing schedules designed by MCTS in the time-frequency domain. Here we use "Hamming weight with a spike" as an example to illustrate how one may design annealing schedule in the time-frequency domain. "Hamming weight with a spike" is commonly used in the comparisons of quantum and classical heuristic optimization algorithms. An insight gained from this problem is that many classical search algorithms often stuck in a local minimum and fail to discover the global minimum while a quantum algorithm can. [11, 12].

The "Hamming weight with a spike" Hamiltonian reads:

$$H_{final} = \sum_{\mathbf{z} \in \{0,1\}^n} c(w) |\mathbf{z}\rangle \langle \mathbf{z}|, \quad (9)$$

where $w = |\mathbf{z}| = |z_1 \dots z_n|$ is the Hamming weight of a n -bit string. $c(w)$ is the potential given by a ramp $r(w) = w$, plus a rectangular "spike" function $s(w)$ centered at $w = n/4$, for two exponents $\alpha, \beta \in [0, 1]$

$$\begin{aligned} \text{Ramp: } r(w) &= w, \text{ Spike: } s(w) = \begin{cases} n^\beta, & \text{if } w \in \left[\frac{n}{4} - \frac{n^\alpha}{2}, \frac{n}{4} + \frac{n^\alpha}{2}\right] \\ 0, & \text{otherwise.} \end{cases} \\ \text{Full Potential: } c(w) &= r(w) + s(w) \end{aligned} \quad (10)$$

In this illustration, we impose $s_0(t)$ to take on a linear schedule for the most part but switch to a bang-bang control for a brief interval at the beginning and the end of the schedule as outlined below,

$$s(t) = \begin{cases} 1 & 0 \leq t < t_1, T - t_3 - t_4 \leq t < T - t_4, \\ 0 & t_1 \leq t < t_1 + t_2, T - t_4 \leq t < T, \\ \frac{t}{T} + \sum_{i=1}^M x_i \sin \frac{i\pi t}{T} & t_1 + t_2 \leq t < T - t_3 - t_4. \end{cases} \quad (11)$$

The optimal control problem is now expanded to assigning values to the sequence $\mathbf{x} = \{t_1, t_2, t_3, t_4, x_1, x_2, x_3, \dots, x_M\}$, where $x_1, x_2, x_3, \dots, x_M$ is the strength of Fourier components and t_1, t_2, t_3, t_4 indicates the distribution of initial/final bang-bang sequences. It is obvious that when $t_i = 0, i = 1, 2, 3, 4$ the evolution schedule is just annealing schedules in the frequency domain.

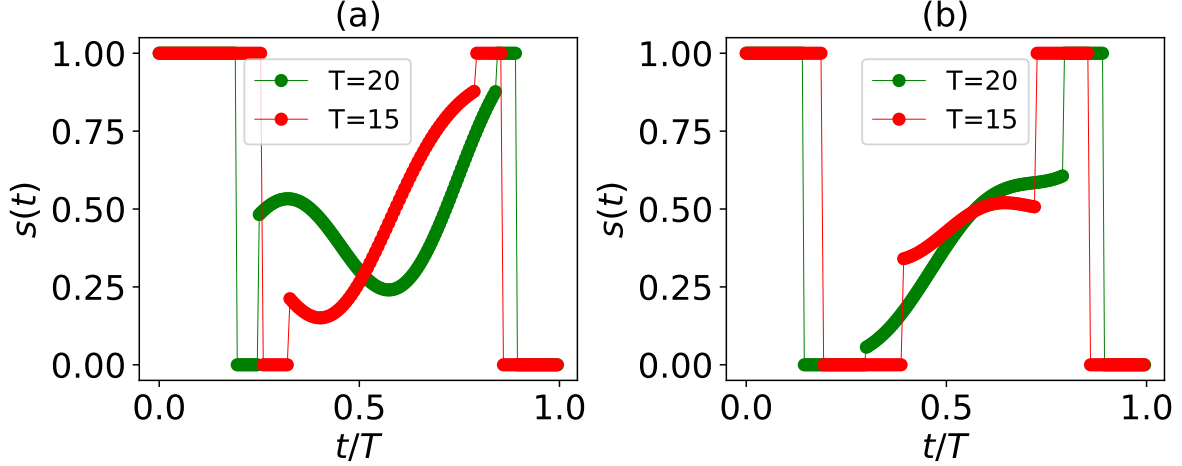


Figure 7: Optimal evolution schedules designed by MCTS. (a) Results of spike instance with $n = 8$, $\alpha = 0.3$, $\beta = 0.2$ for $T = 20, 10$ respectively. (b) Results of spike instance with $n = 8$, $\alpha = 0.3$, $\beta = 0.4$ for $T = 20, 10$ respectively.

Results of MCTS-designed annealing schedules for two Hamming ramp with spike problems are displayed in Fig. 7. Details regarding these Hamiltonians may be found in the caption of the figure. In the first case, Fig. 7(a), the optimal schedules operated under a total time of $T = 20$ and $T = 15$ reach a final fidelity of 0.99 and 0.986 in comparison to the fidelity of 0.91 and 0.908 for a linear path, respectively. In the second instance, shown in Fig. 7(b), the optimal schedules, operated under a total time of $T = 20$ ($T = 15$), reaches a final fidelity of 0.989 and 0.982, which also exceed the fidelity of 0.867 and 0.822 for a linear path, respectively. In all these cases, MCTS recommends schedules with bang-anneal-bang profiles instead of a smooth trajectory.

-
- [1] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller, “Playing atari with deep reinforcement learning,” (2013), [arXiv:1312.5602](#).
 - [2] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis, “Human-level control through deep reinforcement learning,” *Nature* **518**, 529–533 (2015).
 - [3] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba, “Openai gym,” (2016), [arXiv:1606.01540](#).
 - [4] Prafulla Dhariwal, Christopher Hesse, Oleg Klimov, Alex Nichol, Matthias Plappert, Alec Radford, John Schulman, Szymon Sidor, Yuhuai Wu, and Peter Zhokhov, “Openai baselines,” <https://github.com/openai/baselines> (2017).
 - [5] Richard S. Sutton and Andrew G. Barto, *Reinforcement Learning: An Introduction* (A Bradford Book, Cambridge, MA, USA, 2018).
 - [6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347* (2017).
 - [7] J. Roland and N. J. Cerf, “Quantum search by local adiabatic evolution,” *Physical Review A*, vol. 65, no. 4, p. 042308, 2002.
 - [8] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann, “A quantum approximate optimization algorithm,” (2014), [arXiv:1411.4028](#).
 - [9] Seth Lloyd, “Quantum approximate optimization is computationally universal,” (2018), [arXiv:1812.11075](#).
 - [10] Mauro E. S. Morales, Jacob Biamonte, and Zoltán Zimborás, “On the universality of the quantum approximate optimization algorithm,” (2019), [arXiv:1909.03123](#).

- [11] E. Farhi, J. Goldstone, and S. Gutmann, “Quantum adiabatic evolution algorithms versus simulated annealing,” *arXiv preprint quant-ph/0201031*, 2002.
- [12] L. Kong and E. Crosson, “The performance of the quantum adiabatic algorithm on spike hamiltonians,” *International Journal of Quantum Information*, vol. 15, no. 02, p. 1750011, 2017.